

FAANG Master Guide: SPEC PLAN TASKS

Objetivo: No memorizar solues. Entender como reconhecer padres e escolher a melhor abordagem para cada problema.

Framework SPEC PLAN TASKS

- SPEC:** O QU? (Definir o problema)
- Contrato Input/Output
 - Edge cases e restries
 - Propriedades matemticas (monotonicidade?)
 - Fora bruta naive (sempre comece aqui!)
- PLAN:** COMO? (Escolher a tcnica)
- Reconhecer padro (BFS? DP? Greedy? Binary Search?)
 - Comparar trade-offs (Time vs Space vs Legibilidade)
 - Justificar escolha (por qu THIS padro?)
 - Pseudocdigo e diagrama
- TASKS:** CDIGO (Implementar + Validar)
- Cdigo limpo e comentado
 - Testes (edge cases, casos normais ...)
 - Anlise de complexidade (Time + Space)
 - Dbug (printf-debugging, trace mental)

SPEC: Checklist Detalhado

1 Contrato Input/Output

Aspecto	Pergunta	Exemplo
Tipos de entrada	List? Dict? Tree? Graph? String?	nums: List[int], intervals: List[List[int]]

2 Edge Cases Crticos

Sempre teste esses casos:

Tamanho

[], [1], [1, 2] # Vazio, singleton, pair

Valores duplicados

[1, 1, 1], [1, 2, 1] # Todos iguais, repeties

Extremos

[-10^9, 0, 10^9] # Min, zero, max

Casos especiais do domnio

Strings: "", strings com espaos, caracteres especiais

Arrays: Arrays j ordenados, inversamente ordenados

Grafos: Grafo desconexo, n isolado, ciclos

3 Propriedades Matemticas

Pergunta-chave: Qual propriedade do problema ajuda a resolver?

| Propriedade | Implicao | Padro Sugerido |

|-----|-----|-----|

| Monotonicidade | Se $f(k)$ funciona, $f(k+1)$ funciona? | Binary Search |

| Subestrutura tima | Soluo tima = combina solues timas dos subproblemas? | DP |

| Indeciso simples | Preciso explorar TODAS as possibilidades? | Backtracking, DFS |

| Estrutura linear | Array/String processados da esquerda para direita? | Sliding Window, DP 1D |

PLAN: Matriz de Trade-offs

Quando usar cada tcnica?

Tcnica	Time	Space	Quando usar
FORCE BRUTE (tenta tudo)	$O(n!)$ $O(2^n)$	$O(1)$ aqui! $n < 20$	Sempre comece
BACKTRACKING + PRUNING (fora bruta smart) (melhor) stack	$O(n!)$ $O(2^n)$	$O(n)$	Combinaes, Permutaes, Sudoku
DYNAMIC PROGRAMMING (memoization/tabulation) (dep) (rolling) de subproblemas	$O(n)$ $O(n)$	$O(n)$ $O(n)$	Subestrutura tima, repetio
DFS (graph/tree) ou $O(n)$ recurso stack	$O(V+E)$	$O(V)$	Explorar profundidade, Connected comps
BFS (graph/tree) ou $O(n)$ queue	$O(V+E)$	$O(V)$	Menor caminho, Nvel a nvel
BINARY SEARCH (on answer/value) ordenaol)	$O(\log n)$	$O(1)$	Funo monotnica (no sobre
GREEDY $O(n \log n)$ $O(n)$	$O(n)$	$O(1)$	Greedy choice provvel tima (sort)
SLIDING WINDOW (hash) substrings	$O(n)$	$O(k)$	Subarrays/
HASH MAP / SET $O(n)$ bad	$O(n)$ avg Frequencia	$O(n)$	Lookup rpido,
HEAP / PRIORITY QUEUE	$O(n \log n)$	$O(n)$	Top K, Mediana Merge K
MONOTONIC STACK (Deque/Stack simples)	$O(n)$ stack	$O(n)$	Prximo elemento MAIOR/MENOR, Daily Temps

Exemplos de Trade-offs

| Abordagem | Time | Space | Vive!? | Razo |

|-----|-----|-----|-----|

| Linear scan | $O(n)$ | $O(1)$ | Sim | Sempre funciona, mas lento |

| Binary Search | $O(\log n)$ | $O(1)$ | MELHOR | Array est sorted propriedade monotnica |

Concluso: Use Binary Search porque o array sorted monotonicidade.

| Abordagem | Time | Space | Vive!? | Razo |

|-----|-----|-----|-----|

| Brute force (try all) | $O(n!n)$ | $O(n)$ | Horvél | Timeout para $n > 8$ |

| Backtracking + pruning | $O(n!)$ | $O(n)$ | MELHOR | Prune ramos inviveis early |

| DP | No aplicvel | - | No | Sem subestrutura tima |

****Concluso:**** Use Backtracking porque precisamos explorar TODAS as combinaes + poder fazer pruning.

| Abordagem | Time | Space | Vive!? | Razo |

|-----|-----|-----|-----|

| Greedy (maior moeda first) | $O(n)$ | $O(1)$ | No | No sempre timo (ex: [1,3,4], value=6) |

| DFS

| DP (memoization) | $O(nk)$ | $O(n)$ | MELHOR | Subestrutura tima: $\min(\text{coin1..k}) + \min(\text{rest})$ |

****Concluso:**** Use DP porque temos subestrutura tima + repetio de subproblemas.

Decision Tree: Como Escolher?

NOVO PROBLEMA

[1] um problema de GRAFO/RVORE?

YES BFS (shortest path) ou DFS (conectividade)?

Topological sort? Dijkstra?

NO [2]

[2] Preciso ENCONTRAR TODAS as solues (combinaes/permutaes)?

YES BACKTRACKING + PRUNING (N-Queens, Sudoku)

NO [3]

[3] um problema de OTIMIZAO (min/max)?

YES [4a]

NO [5]

[4a] A funo objetivo MONOTNICA?

YES BINARY SEARCH ON ANSWER (Koko Bananas)

IMPORTANTE: Monotonicidade de FUNO (curva sempre sobe/desce)

NO confundir com MONOTONIC STACK (estrutura de dados)!

Monotonic Stack para problemas como:

- Nearest Greater/Smaller Element

- Daily Temperatures

- Trapping Rain Water

Use Monotonic Stack quando precisa encontrar padres

de vizinhana em arrays (no otimizao com busca)

NO [4b]

[4b] Tem SUBESTRUTURA TIMA (soluo tima = $\text{opt}(\text{subproblemas})$)?

YES DYNAMIC PROGRAMMING (Coin Change, LIS, House Robber)

NO GREEDY (Jump Game, Interval Scheduling)

Greedy nem sempre funciona! Prove ou valide com testes.

[5] um SUBSTRING/SUBARRAY problem?

YES SLIDING WINDOW (Fixed/Variable size)

ou TWO POINTERS (contraste entre elementos)

NO [6]

[6] Frequncia, rank, top K?

YES HEAP / HASH MAP (K Frequent, Kth Largest)

NO [7]

[7] Array/String simples?

YES TWO POINTERS, SORT + GREEDY, ou HASH MAP

NO Check outras propriedades (linked list? matrix?)

Padres de Reconhecimento

1. ****Quando reconhecer DP?****

GATILHOS DP:

"mnimo", "mximo" (otimizao)

"contar", "nmero de formas" (combinatria)

"possvel?", "vivel?" (deciso)

Overlapping subproblems (fibonacci, coin change)

Subestrutura tima (a resposta usa respostas menores)

Frases que triggerem DP:

- "Minimum steps to..."

- "Maximum product of..."

- "Number of ways to..."

- "Can we achieve...?"

- "Is it possible...?"

Exemplo: "Mnimo nmero de moedas para fazer X"

Subestrutura: $\text{min_coins}(X) = 1 + \min(\text{min_coins}(X - \text{coin}) \text{ for each coin})$

2. **Quando reconhecer Backtracking?**

GATILHOS BACKTRACKING:

"todas as combinaes" / "todas as permutaes"

"encontrar todas as solues"

"escolher/descartar elementos"

Decises independentes em cada nvel

Frases que triggerem Backtracking:

- "Find ALL combinations/permutations"

- "Generate all valid..."

- "Solve the puzzle" (Sudoku, N-Queens)

- "All possible ways to..."

Exemplo: "Todas as permutaes de uma string"

Escolha: cada posio pode ter qualquer caractere ainda no usado.

2 **O que Monotonicidade? (Conceito Essencial)**

Monotonicidade a propriedade de uma **FUNO**, no dos dados!

Funo MONOTNICA (crescente ou decrescente) significa que:

CRESCENTE MONTONA:

Se $x < x$, ento $f(x) \leq f(x)$ (ou $<$ para estritamente crescente)

Exemplo: $f(x) = x$, $f(x) = 2x + 1$, $f(x) = x^2$

DECRESCENTE MONTONA:

Se $x < x$, ento $f(x) \geq f(x)$ (ou $>$ para estritamente decrescente)

Exemplo: $f(x) = -x$, $f(x) = 1/x$, $f(x) = -2x + 5$

MONOTNICA: Sempre vai na MESMA DIREO

Sempre cresce: $f(1) < f(2) < f(3) < f(4) \dots$

Sempre decresce: $f(1) > f(2) > f(3) > f(4) \dots$

Nunca muda de direo!

NO MONOTNICA: Muda de direo

Cresce e depois decresce: $f(1) < f(2) < f(3) > f(4)$

Decresce e depois cresce: $f(1) > f(2) > f(3) < f(4)$

CRESCENTE MONTONA ($f(x) = x$):

y
|
|
|
|
|
|
x

Padro: Sempre SOBE (ou fica igual)

Logo: $f(1) < f(2) < f(3) < f(4)$

DECRESCENTE MONTONA ($f(x) = -x$):

y
|
|
|
|

|
|
x

Padro: Sempre DESCE (ou fica igual)
Logo: $f(1) > f(2) > f(3) > f(4)$

NO MONOTNICA ($f(x) = x - 4x$):

y
|
|
|
|
|
|
x

Padro: Desce, depois SOBE
Logo: NO monotnica! (muda de direo)

PROBLEMA: Procurar um valor em um espao grande $[1, 10^9]$

Com MONOTONICIDADE (f sempre cresce ou sempre decresce):

Se estou no meio e $f(\text{mid}) > \text{target}$:

Posso ignorar METADE do espao com segurana!

Porque sei que no vai mudar de direo

Logo: $O(\log n)$ garantido!

Sem MONOTONICIDADE (f sobe e desce):

Se estou no meio e $f(\text{mid}) > \text{target}$:

No sei se ignorar esquerda ou direita!

A resposta pode estar em QUALQUER LUGAR

Logo: Preciso de fora bruta $O(n)$

Pilhas: [3, 6, 7, 11]

Horas disponveis: 8

Funco: $\text{hours_needed}(k) = \sum(\text{ceil}(\text{pile} / k))$

Valores:

$k=1$ $\text{hours_needed}(1) = 4 + 6 + 7 + 11 = 28$

$k=2$ $\text{hours_needed}(2) = 2 + 3 + 4 + 6 = 15$

$k=3$ $\text{hours_needed}(3) = 2 + 2 + 3 + 4 = 11$

$k=4$ $\text{hours_needed}(4) = 1 + 2 + 2 + 3 = 8$ resposta!

$k=5$ $\text{hours_needed}(5) = 1 + 2 + 2 + 3 = 8$

$k=6$ $\text{hours_needed}(6) = 1 + 1 + 2 + 2 = 6$

$k=7$ $\text{hours_needed}(7) = 1 + 1 + 1 + 2 = 5$

Propriedade: $28 > 15 > 11 > 8$ $8 > 6 > 5$

SEMPRE DECRESCENTE (ou igual)

MONOTNICA! Logo Binary Search funciona.

Funco: $f(x) = x^2 - 8x + 12$ (parbola)

Valores:

$x=0$ $f(0) = 12$

$x=1$ $f(1) = 5$

$x=2$ $f(2) = 0$

$x=3$ $f(3) = -3$

$x=4$ $f(4) = -4$ mnimo

$x=5$ $f(5) = -3$

$x=6$ $f(6) = 0$

$x=7$ $f(7) = 5$

Propriedade: $12 > 5 > 0 > -3 > -4 < -3 < 0 < 5$

Desce depois SOBE!

NO MONOTNICA! Binary Search NO funciona diretamente.

(Precisaria de tcnica especial para encontrar o mnimo)

3. **Quando reconhecer Binary Search?**

GATILHOS BINARY SEARCH:

Funco MONOTNICA: $f(x) = [F, F, \dots, F, T, T, \dots, T]$ ou $[T, T, \dots, T, F, F]$

"Mnimo X tal que..." ou "Mximo X tal que..."

Array sorted OU espao de busca bem definido

Constraints: n at 10^{18} (log n vive!, linear no)

IMPORTANTE: Monotonicidade NO ordenao!

Monotonicidade = propriedade da FUNO (input output ou)

Ordenao = estado dos DADOS (array em ordem crescente)

Exemplo Koko: [3,6,7,11] NO sorted, mas horas_needed(k) monotnica

Frases que triggerem Binary Search:

- "Minimum/Maximum [X] such that [condition]"

- "Can we do it in [X] time/capacity?"

- "First/Last position of..."

Exemplo: "Mnimo velocidade para comer todas as bananas" (Koko)

Monotonicidade: se velocidade V funciona, V+1 tambm funciona.

4 ****Quando reconhecer Monotonic Stack? (NO confundir com Binary Search!)****

ALERTA CONFUSO:

Muitas pessoas confundem "Monotnica" (Binary Search)

com "Monotonic Stack" (estrutura de dados)

SO COISAS COMPLETAMENTE DIFERENTES!

Monotonic Stack uma ESTRUTURA DE DADOS (no uma propriedade de funo!)

Mantm elementos em ordem CRESCENTE ou DECRESCENTE

Usando: Deque ou Stack com pop() estratgico

Exemplo:

Input: [73, 74, 75, 71, 69, 72, 76, 73]

Problema: Daily Temperatures - prxima temperatura MAIOR

Stack (monotonic decreasing):

[73] Stack vazio, push

[73, 74] 74 > 73, pop 73, encontra resposta! push 74

[73, 74, 75] 75 > 74, pop 74, push 75

[73, 74, 75, 71] 71 < 75, push (sem pop)

...

"Prximo elemento MAIOR/MENOR"

"Elemento anterior MAIOR/MENOR"

"Nmero de dias at temperatura maior"

"Elemento com maior valor direita/esquerda"

"Remover k dgitos para fazer nmero menor"

"Largest Rectangle in Histogram"

"Trapping Rain Water"

Padro: Precisa encontrar padres de vizinhana/relao entre elementos

No otimizao com busca (ento NO Binary Search)

Complexidade: $O(n)$ - cada elemento entra/sai da stack UMA VEZ

Aspecto	MONOTNICA (Binary)	MONOTONIC STACK
---------	--------------------	-----------------

O que	Propriedade de funo	Estrutura de dados
-------	---------------------	--------------------

Pergunta tpica	"Mnimo X tal que"	"Prximo X MAIOR"
----------------	-------------------	------------------

Estrutura	Espao de busca	Stack/Deque
-----------	----------------	-------------

Algoritmo	Binary Search	Linear scan com pop()
-----------	---------------	-----------------------

Complexidade	$O(\log n) + O(n)$	$O(n)$
--------------	--------------------	--------

Exemplo	Koko Bananas	Daily Temperatures
---------	--------------	--------------------

Usa comparao mid com cond?	SIM	NO
----------------------------	-----	----

Precisa manter ordem?	NO (funo vale)	SIM (stack ordem)
-----------------------	----------------	-------------------

Daily Temperatures: [73, 74, 75, 71, 69, 72, 76, 73]

Output: Dias at prxima temperatura MAIOR

```
def dailyTemperatures(temps):
```

```
    result = [0] * len(temps)
```

```
    stack = [] # Vai guardar NDICES, mantendo temperaturas em ordem DECRESCENTE
```

```
    for i, temp in enumerate(temps):
```

```
        # Enquanto stack no vazio E temperatura atual > topo do stack
```

```

while stack and temp > temps[stack[-1]]:
j = stack.pop()
result[j] = i - j # Encontrou! A diferenca a resposta
stack.append(i)
return result

```

Trace:

temps = [73, 74, 75, 71, 69, 72, 76, 73]

i=0, temp=73: stack=[] push(0) stack=[0]

i=1, temp=74: 74 > 73? SIM! pop(0), result[0]=1-0=1, push(1) stack=[1]

i=2, temp=75: 75 > 74? SIM! pop(1), result[1]=2-1=1, push(2) stack=[2]

i=3, temp=71: 71 > 75? NO push(3) stack=[2,3]

i=4, temp=69: 69 > 71? NO push(4) stack=[2,3,4]

i=5, temp=72: 72 > 69? SIM! pop(4), result[4]=5-4=1

72 > 71? SIM! pop(3), result[3]=5-3=2

72 > 75? NO push(5) stack=[2,5]

i=6, temp=76: 76 > 72? SIM! pop(5), result[5]=6-5=1

76 > 75? SIM! pop(2), result[2]=6-2=4, push(6) stack=[6]

i=7, temp=73: 73 > 76? NO push(7) stack=[6,7]

result = [1, 1, 4, 2, 1, 1, 0, 0]

5. **Quando reconhecer Greedy?**

GATILHOS GREEDY:

Problema de otimizacao (min/max)

Escolha LOCAL timo leva a timo GLOBAL

No h subestrutura timo ou bvio

AVISO: Greedy nem sempre funciona!

Coin Change com [1, 3, 4] e valor 6:

Greedy: 4 + 1 + 1 = 3 moedas

tima: 3 + 3 = 2 moedas

Validao Greedy:

1. Prove que escolha local timo (prova matemtica)
2. OU teste exaustivamente com pequenos exemplos

3. OU use DP se no tiver certeza

Exemplos do Repositrio

Jump Game (Greedy)

****SPEC:****

- Input: `nums: List[int]` (alcance mximo de cada posio)
- Output: `bool` (consegue chegar ao final?)
- Edge: `[]`, `[0]`, `[2,3,1,1,4]`
- Propriedade: Subestrutura tima? SIM (pode alcanar i pode alcanar j < i e j+nums[j] >= i)

****PLAN:****

- Fora bruta: DFS all paths $O(2^n)$
- DP: `dp[i] = True` se conseguir alcanar i $O(n)$
- Greedy: `max\_reach` track $O(n)$ ****MELHOR****

****CDIGO:****

```
def canJump(nums: List[int]) -> bool:
    """Greedy: acompanha mximo alcance."""
    max_reach = 0
    for i in range(len(nums)):
        if i > max_reach:
            return False # No consegue alcanar posio i
        max_reach = max(max_reach, i + nums[i])
    return True
```

Tests

```
assert canJump([2, 3, 1, 1, 4]) == True # Consegue
assert canJump([3, 2, 1, 0, 4]) == False # Cai no 0
assert canJump([0]) == True # J no final
assert canJump([2, 0, 0]) == True # Pode alcanar
print(" Jump Game I Passed")
```

Escolha: ****Greedy**** porque propriedade especial: monotonicidade no alcance.

Jump Game II (BFS - Nvel a Nvel)

****SPEC:****

- Input: `nums: List[int]` (alcance mximo de cada posio)
- Output: `int` (nmero MNIMO de jumps para chegar ao final)
- Restrio: Sempre consegue chegar (dados garantem)
- Propriedade: ****Nveis**** (cada jump 1 nvel)

****PLAN:****

- Fora bruta: DFS explorando todos os jumps $O(2^n)$
- DP: `dp[i] = min jumps para alcanar i` $O(n)$
- BFS: Processa nvel a nvel, contabiliza jumps $O(n)$ ****MELHOR****

****Intuio BFS:****

Cada iterao processa 1 nvel de BFS
Quando termina de processar um nvel, jumps += 1

****CDIGO:****

```
def jump(nums: List[int]) -> int:
    """BFS nvel a nvel: cada nvel = 1 jump."""
    jumps = 0
    current_end = 0 # Fim do nvel atual
    farthest = 0 # Mximo alcanado
    for i in range(len(nums) - 1): # No precisa processar ltimo
        farthest = max(farthest, i + nums[i])
    # Terminou este nvel? (i == current_end)
    if i == current_end:
        jumps += 1
        current_end = farthest
    return jumps
```

Tests

```
assert jump([2, 3, 1, 1, 4]) == 2    # [0,1,4] = 2 jumps
assert jump([2, 3, 0, 1, 4]) == 2    # [0,1,4] = 2 jumps
assert jump([1, 1, 1, 0]) == 3       # [0,1,2,3] = 3 jumps
assert jump([10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 1, 0]) == 2 # [0, ...] = 2
print(" Jump Game II Passed")
```

****Trade-off:****

- DP O(n) correto mas redundo
- BFS O(n) natural: simula os jumps como nveis do grafo

Jump Game III (DFS/BFS com Ciclos)

****SPEC:****

- Input: `arr: List[int]` (array), `start: int` (posio inicial)
- Output: `bool` (consegue alcanar 0?)
- Restrio: ****Pode pular PARA FRENTE E PARA TRS**** (pode ter ciclos!)
- Edge: `[4,2,3,0,3,1,2]`, start=5

****PLAN:****

- DFS sem visited: Ciclo infinito!
- DFS com visited: Marca visitados, evita ciclos O(n) ****MELHOR****
- BFS com visited: Similar, tambm funciona

****CDIGO - DFS Version:****

```
def canReach(arr: List[int], start: int) -> bool:
    """DFS com visited set: evita ciclos infinitos."""
    visited = set()
```

```
    def dfs(i):
```

```
        # Validao
```

```
        if i < 0 or i >= len(arr) or i in visited:
```

```
            return False
```

```
        # Chegou no 0!
```

```
        if arr[i] == 0:
```

```
            return True
```

```
        # Marca como visitado (ANTES de explorar filhos)
```

```
        visited.add(i)
```

```
        # Pode pular FRENTE E TRS
```

```
        return dfs(i + arr[i]) or dfs(i - arr[i])
```

```
    return dfs(start)
```

Tests

```
assert canReach([4, 2, 3, 0, 3, 1, 2], 5) == True    # 51430
assert canReach([3, 0, 2, 1, 2], 2) == False        # No consegue
assert canReach([0], 0) == True                     # J no 0
assert canReach([1, 0], 1) == True                  # 10
print(" Jump Game III (DFS) Passed")
```

****CDIGO - BFS Version:****

```
from collections import deque
```

```
def canReach_BFS(arr: List[int], start: int) -> bool:
```

```
    """BFS com visited set: nvel a nvel."""
```

```
    visited = set([start])
```

```
    queue = deque([start])
```

```
    while queue:
```

```
        i = queue.popleft()
```

```
        if arr[i] == 0:
```

```
            return True
```

```
        # Explore vizinhos (frente e trs)
```

```
        for next_i in [i + arr[i], i - arr[i]]:
```

```
            if 0 <= next_i < len(arr) and next_i not in visited:
```

```
                visited.add(next_i)
```

```
                queue.append(next_i)
```

```
return False
```

Tests (mesmos)

```
assert canReach_BFS([4, 2, 3, 0, 3, 1, 2], 5) == True
print(" Jump Game III (BFS) Passed")

**Comparao Jump Game I/II/III:**
| Problema | Direo | Ciclos? | Padro | Complexidade |
|-----|-----|-----|-----|-----|
| Jump I | Frente | No | Greedy | O(n) |
| Jump II | Frente | No | BFS | O(n) |
| Jump III | Frente + Trs | SIM | DFS/BFS+visited | O(n) |

---
```

Koko Eating Bananas (Binary Search)

```
**SPEC:**
- Input: `piles: List[int]`, `h: int`
- Output: `int` (mnimo velocidade)
- Propriedade: **Monotonicidade** (se k funciona, k+1 funciona)

**PLAN:**
- Search space: `[1, max(piles)]`
- Binary Search encontra primeiro k onde `hours_needed(k) <= h`

**Trade-off:**
- Brute force: `for k in range(1, max(piles))` O(n max)
- Binary search O(n log(max)) **MELHOR**

** IMPORTANTE: Leia a seo "TASKS" abaixo para entender COMPLETAMENTE!**

---
```

N-Queens (Backtracking)

```
**SPEC:**
- Input: `n: int` (tamanho do tabuleiro)
- Output: `List[List[str]]` (todas as solues)
- Precisa: TODAS as solues

**PLAN:**
- Fora bruta: Tentar todas as colocaes de N rainhas O(N!) (sem pruning)
- Backtracking: Prunar posies invlidas O(N!) (com pruning)

**Trade-off:**
- DP no aplica (sem subestrutura tima)
- BFS/DFS no usa bem (problema combinatrio)
- **Backtracking** natural

**CDIGO:**
def solveNQueens(n: int) -> List[List[str]]:
    """Backtracking: coloca rainhas linha por linha com pruning."""
    result = []
    board = [['.' for _ in range(n)] for _ in range(n)]
    cols, diag1, diag2 = set(), set(), set()
    # diag1: i - j (mesma anti-diagonal)
    # diag2: i + j (mesma diagonal)

    def backtrack(row):
        # Terminou: colocou n rainhas
        if row == n:
            result.append([''.join(line) for line in board])
            return

        # Tenta colocar rainha em cada coluna desta linha
        for col in range(n):
            # Validao: no tem rainha nesta:
            # - coluna
            # - diagonal (i - j)
            # - anti-diagonal (i + j)
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue # Poda: posio invlida
```

```

# Coloca rainha
board[row][col] = 'Q'
cols.add(col)
diag1.add(row - col)
diag2.add(row + col)

# Recurso: proxima linha
backtrack(row + 1)

# BACKTRACK: desfaz colocao
board[row][col] = '.'
cols.remove(col)
diag1.remove(row - col)
diag2.remove(row + col)

backtrack(0)
return result

```

Tests

```

result_4 = solveNQueens(4)
assert len(result_4) == 2      # 4-Queens tem 2 solues
assert all(len(sol) == 4 for sol in result_4)

result_1 = solveNQueens(1)
assert len(result_1) == 1      # 1-Queens tem 1 soluo
assert result_1[0][0] == "Q"

print(" N-Queens Passed")

**Complexidade:**
- Time: O(N!) - no pior caso explora todas as permutaes
Mas com pruning, muito mais rpido na prtica
- Space: O(N) - recursion stack depth + sets
---
```

Container With Most Water (Two Pointers)

```

**SPEC:**
- Input: `heights: List[int]` (altura de cada barra vertical)
- Output: `int` (mxima rea de gua)
- Edge: `[]`, `[1, 1]`, alturas podem ser 0
- Propriedade: **Greedy + contraste** (duas pontas convergem)

**PLAN:**
- Fora bruta: Todos os pares (i, j) O(n)
- Two Pointers: Comea nas extremidades, move o menor O(n) **MELHOR**

**Intuio Greedy:**
rea = min(height[left], height[right]) (right - left)
Se move o MAIOR distncia diminui, altura no aumenta muito
Se move o MENOR chance de encontrar altura maior (rea cresce)

**CDIGO:**
def maxArea(heights: List[int]) -> int:
    """Two Pointers: comea nas extremidades, move o menor."""
    left, right = 0, len(heights) - 1
    max_area = 0

    while left < right:
        # Calcula rea atual
        width = right - left
        current_area = min(heights[left], heights[right]) * width
        max_area = max(max_area, current_area)

        # Move o ponteiro da barra MENOR
        # (esperana: encontrar barra maior)
        if heights[left] < heights[right]:
            left += 1
        else:
            right -= 1

    return max_area

```

Tests

```

assert maxArea([1, 8, 6, 2, 5, 4, 8, 3, 7]) == 49 # Entre ndices 1 e 8
assert maxArea([1, 1]) == 1 # Mnimo
assert maxArea([4, 3, 2, 1, 4]) == 16 # Extremidades
assert maxArea([2, 3, 4, 5, 18, 17, 6]) == 17 # Meio
print(" Container With Most Water Passed")

**Trade-off:**
- Fora bruta O(n) correto mas lento
- Two Pointers O(n) explora greedy: mover o menor sempre timo para prxima iterao

**Complexidade:**
- Time: O(n) - dois ponteiros passam uma vez
- Space: O(1) - apenas variveis
---
```

Longest Repeating Character Replacement (Sliding Window)

```

**SPEC:**
- Input: `s: str`, `k: int` (mximo de replacements)
- Output: `int` (mximo length de substring com mesmo caractere)
- Propriedade: **Janela varivel** (expandir/contrair)

**PLAN:**
- Fora bruta: Todas as substrings, contar replacements O(n)
- Sliding Window: Manter janela onde (len - max_freq) <= k O(n) **MELHOR**

**Lgica:**
window_len = right - left + 1
chars_to_replace = window_len - max_frequency_in_window
Se chars_to_replace <= k: janela vlida, expanda
Seno: contraia (move left)

**CDIGO:**
def characterReplacement(s: str, k: int) -> int:
    """Sliding Window: mantm janela com at k replacements."""
    from collections import Counter

    left = 0
    char_count = Counter()
    max_freq = 0
    max_length = 0

    for right in range(len(s)):
        # Expande janela
        char_count[s[right]] += 1
        max_freq = max(max_freq, char_count[s[right]])

        # Valida janela
        window_len = right - left + 1
        chars_to_replace = window_len - max_freq

        # Se precisa trocar mais de k caracteres, contrai
        if chars_to_replace > k:
            char_count[s[left]] -= 1
            left += 1

        # Atualiza resposta
        max_length = max(max_length, right - left + 1)

    return max_length
```

Tests

```

assert characterReplacement("ABAB", 2) == 4 # Troca 2 B's "AAAA"
assert characterReplacement("ABBB", 2) == 4 # Troca 2 A's "BBBB"
assert characterReplacement("AABAB", 1) == 3 # "AAA" ou "BBB"
assert characterReplacement("A", 0) == 1 # Sem replacements
print(" Longest Repeating Character Replacement Passed")

**Trade-off:**
- DP com memoization O(n2) correto mas overkill
- Sliding Window O(n) com hash map natural e elegante

**Complexidade:**
- Time: O(n) - cada caractere processado uma vez
- Space: O(26) = O(1) - no mximo 26 letras
```

Word Search / Trie (DFS Backtracking)

****SPEC:****

- Input: `board: List[List[str]]`, `word: str`
- Output: `bool` (palavra existe no tabuleiro?)
- Restrio: Pode se mover up/down/left/right, cada clula usa apenas 1x
- Propriedade: ****Grafo com backtracking**** (explora caminhos, marca visitado)

****PLAN:****

- BFS/DFS com visited set: Marca clula visitada durante busca
- Backtracking: Unmark ao retornar (permite reexplorao em outros caminhos)

****Lgica DFS:****

```
def dfs(i, j, idx):
    # Base: encontrou toda a palavra
    if idx == len(word):
        return True

    # Validao
    if i < 0 or j < 0 or i >= rows or j >= cols:
        return False
    if board[i][j] != word[idx] or (i, j) in visited:
        return False

    # Backtrack
    visited.add((i, j))
    for di, dj in [(0,1), (0,-1), (1,0), (-1,0)]:
        if dfs(i+di, j+dj, idx+1):
            return True

    visited.remove((i, j)) # CRUCIAL: remove ao retornar
    return False
```

****CDIGO:****

```
def exist(board: List[List[str]], word: str) -> bool:
    """DFS com backtracking: marca visitado, depois desmarcar."""
    if not board or not word:
        return False

    rows, cols = len(board), len(board[0])
    visited = set()

    def dfs(i, j, idx):
        # Terminou a palavra?
        if idx == len(word):
            return True

        # Validao de limites
        if i < 0 or i >= rows or j < 0 or j >= cols:
            return False

        # Validao de caractere e visitado
        if board[i][j] != word[idx] or (i, j) in visited:
            return False

        # MARCA COMO VISITADO
        visited.add((i, j))

        # Explora 4 direes
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
        for di, dj in directions:
            if dfs(i + di, j + dj, idx + 1):
                # ENCONTROU! Pode retornar sem limpar
                return True

        # BACKTRACK: remove do visitado para outras exploraes
        visited.remove((i, j))
        return False

    # Tenta comear de cada posio
    for i in range(rows):
        for j in range(cols):
            if board[i][j] == word[0] and dfs(i, j, 0):
                return True

    return False
```

Tests

```
board1 = [["O", "A", "A"], ["A", "A", "A"], ["A", "A", "O"]]
assert exist(board1, "AAA") == True      # Mltiplos caminhos
board2 = [["A", "B"], ["A", "A"]]
assert exist(board2, "AAB") == True      # Frente e trs
board3 = [["A", "B"], ["C", "D"]]
assert exist(board3, "ABCD") == False    # No existe
board4 = [["A"]]
assert exist(board4, "A") == True        # Clula nica
print(" Word Search Passed")
**Trade-off:**
- BFS puro no funciona (precisa rastrear caminho + backtrack)
- DFS com backtracking natural para esse tipo de explorao
**Complexidade:**
- Time:  $O(nm4^L)$  onde  $n$ =rows,  $m$ =cols,  $L$ =len(word)
 $4^L$  porque cada clula pode explorar 4 direes no pior caso
- Space:  $O(L)$  - recursion stack (backtracking depth)
---
```

Union-Find / DSU (Disjoint Set Union)

```
**SPEC:**
- Input: `edges: List[List[int]]`, `n: int` (ns)
- Output: `int` (nmero de componentes conectadas) ou `bool` (tem ciclo?)
- Propriedade: **Componentes do grafo no-dirigido**
**PLAN:**
- DFS/BFS clssico: Mark visited  $O(V + E)$  (simples)
- Union-Find (DSU): Merge componentes durante construo  $O(V + E(V))$  (elegante)
**Quando usar DSU?**
Mltiplas queries de conectividade
Detectar ciclos
Mnimum Spanning Tree (Kruskal)
Grafo dinmico (adiciona arestas incrementalmente)
Precisa do caminho exato use DFS/BFS
Grafo direcionado com ordem especifica use Topological Sort
**CDIGO:**
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, x):
        """Encontra raiz com path compression."""
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        """Une dois componentes. Retorna False se j estavam unidos (ciclo!)."""
        px, py = self.find(x), self.find(y)
        if px == py:
            return False # J no mesmo componente (ciclo detectado!)
        # Union by rank: rvore menor aponta para maior
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True # Nova conexo
    def countComponents(n: int, edges: List[List[int]]) -> int:
        """Conta componentes conectadas usando DSU."""
        dsu = DSU(n)
```

```

for u, v in edges:
    dsu.union(u, v)

# Conta quantas razes nicas existem
return len(set(dsu.find(i) for i in range(n)))

def hasRedundantConnection(edges: List[List[int]]) -> List[int]:
    """Detecta ciclo (aresta redundante) em grafo no-dirigido."""
    dsu = DSU(len(edges) + 1)

    for u, v in edges:
        # Se no conseguir unir, porque j estavam conectados (ciclo!)
        if not dsu.union(u, v):
            return [u, v]

    return []

```

Tests

Exemplo 1: 3 componentes conectadas

```

edges1 = [[0, 1], [1, 2], [4, 5]]
assert countComponents(6, edges1) == 3 # {0,1,2}, {3}, {4,5}

```

Exemplo 2: Detecta ciclo

```

edges2 = [[1, 2], [1, 3], [2, 3]]
assert hasRedundantConnection(edges2) == [2, 3] # Ciclo: 1-2-3-1

```

Exemplo 3: Sem ciclo

```

edges3 = [[1, 2], [2, 3], [3, 4]]
assert hasRedundantConnection(edges3) == []
print(" Union-Find / DSU Passed")

**Trade-off:**
- DFS: Mais intuitivo, mais simples para 1-2 queries
- DSU: Melhor para mltiplas queries, mais elegante em Union-Find especifico

**Complexidade:**
- find(): O((n)) O(1) com path compression
- union(): O((n)) O(1) com union by rank
- Total: O(E(n)) O(E)

---
```

Checklist de Implementao (TASKS)

Antes de comear a cdigo:

ESTRUTURA:

Funco assinada com tipos (Python 3.9+)
 Docstring com descrio, exemplo, complexidade
 Funco auxiliar se necessrio (helper function)

CDIGO:

Variveis com nomes descritivos
 Comentrios explicando LGICA, no bvio
 Edge cases tratados ([], None, etc)
 Sem magic numbers (use constantes nomeadas)

VALIDAO:

Casos normais testados
 Edge cases testados
 Stress test (grande entrada)
 Tracear manualmente um exemplo

COMPLEXIDADE:

Time complexity declarado
 Space complexity declarado
 Explicao de por que essa complexidade

Template TASKS

```
from typing import List

def solve_problem(nums: List[int]) -> int:
    """
    Descricao clara do que faz.

    Args:
    nums: Lista de inteiros, 1 <= n <= 10^4

    Returns:
    int: Resultado da operacao

    Time: O(n log n) - sorting + binary search
    Space: O(1) - only constant extra space

    Examples:
    >>> solve_problem([3, 6, 7, 11])
    4
    """

    # SPEC check: validate input
    if not nums:
        return 0

    # PLAN: implement chosen pattern
    result = 0
    # ... main logic ...

    # TASKS: return and validate
    return result
```

Tests

```
if __name__ == "__main__":
    # Normal cases
    assert solve_problem([3, 6, 7, 11]) == 4

    # Edge cases
    assert solve_problem([]) == 0
    assert solve_problem([1]) == 1

    # Stress (large input)
    assert solve_problem(list(range(1000))) >= 0

    print("All tests passed ")

---
```

Referencias Rpidas

Complexidade Assinttica

$O(1)$	Constant (best)
$O(\log n)$	Logarithmic (binary search)
$O(n)$	Linear (simple loop)
$O(n \log n)$	Linearithmic (sorting, merge)
$O(n^2)$	Quadratic (nested loops, DP 2D)
$O(n^3)$	Cubic (rare, avoid)
$O(2^n)$	Exponential (backtracking, brute force)
$O(n!)$	Factorial (permutations, very bad)

Space Complexity

$O(1)$	Constant, no extra space
$O(\log n)$	Recursion stack depth
$O(n)$	Array/Hash storage
$O(n^2)$	2D DP table

Resumo: No Decore, Escolha!

REGRA DE OURO PARA NO DECORAR:

1. SPEC: Entenda TODAS as constraints do problema
2. PLAN: Reconhea qual PROPRIEDADE o problema tem
3. TASKS: Implemente a solucao que EXPLORA essa prop.

Exemplo: Problema tem subestrutura tima?

DP natural (no decorao)

Exemplo: Array sorted?

Binary Search natural (no decorao)

Exemplo Completo: Koko Eating Bananas

SPEC

Problema: Koko adora bananas. H `n` pilhas. Cada pilha `i` tem `piles[i]` bananas. Ela tem `h` horas. A cada hora, ela escolhe UMA pilha e come `k` bananas daquela pilha. Se a pilha tiver menos de `k`, ela come tudo. Qual o **mnimo `k`** para comer todas as bananas em `h` horas?

- **Input:** `piles: List[int]` (bananas por pilha), `h: int` (horas disponveis)

- **Output:** `int` (mnima velocidade em bananas/hora)

- **Edge:** `[ ]`, `[1]`, `[10^9]`, `h = 1` (precisa comer tudo em 1 hora!)

- **Propriedade CRTICA:** **Monotonicidade** se consegue com `k=5`, consegue com `k=6` tambm!

PLAN

1. Preciso OTIMIZAR (encontrar mnimo)

2. A funo objetivo MONOTNICA?

hours_needed(k=1) = 4+6+7+11 = 28 horas lento

hours_needed(k=2) = 2+3+4+6 = 15 horas mais rpido

hours_needed(k=3) = 2+2+3+4 = 11 horas ainda melhor

hours_needed(k=4) = 1+2+2+3 = 8 horas timo! (k=4)

hours_needed(k=5) = 1+2+2+3 = 8 horas mesmo que k=4

Padro: CONFORME k AUMENTA hours_needed(k) DIMINUI
(Funo DECRESCENTE = MONOTNICA!)

3. Logo: Binary Search consegue encontrar o mnimo k!

Pilha com 11 bananas, velocidade k:

k=1: 11 horas (1 banana/hora 11 bananas)

k=2: ceil(11/2) = ceil(5.5) = 6 horas

(Hora 1-5: 2 ban/hora = 10 bananas)

(Hora 6: 1 banana restante, ainda conta 1 hora!)

k=3: ceil(11/3) = ceil(3.67) = 4 horas

(Hora 1-3: 3 ban/hora = 9 bananas)

(Hora 4: 2 bananas restantes, ainda conta 1 hora!)

k=4: ceil(11/4) = ceil(2.75) = 3 horas

NUNCA FAA: 11 / 4 = 2.75 horas (ERRADO!)

A gente precisa ARREDONDAR PARA CIMA porque:

- Se sobrar nem que seja 1 banana, precisa de 1 hora inteira!

ERRADO - Float impreciso para nmeros grandes

```
import math
```

```
horas = math.ceil(11 / 4) # Funciona, mas perde preciso em 10^9
```

CERTO - Integer division sem float

```
horas = (11 + 4 - 1) // 4 # = 14 // 4 = 3
```

```
horas = (pile + k - 1) // k # Frmula universal!
```

Prova: Por que (a + b - 1) // b funciona?

Caso 1: $a \% b == 0$ (diviso exata)

$(12 + 3 - 1) // 3 = 14 // 3 = 4$ (correto: $12 / 3 = 4$)

Caso 2: $a \% b != 0$ (tem resto)

$(11 + 3 - 1) // 3 = 13 // 3 = 4$ (correto: $\text{ceil}(11/3) = 4$)

Objetivo: Encontrar MNIMO k onde $\text{hours_needed}(k) \leq h$

$\text{min_speed} = 1$ velocidade mnima possvel

$\text{max_speed} = \text{max}(\text{piles})$ velocidade mxima (come uma pilha por hora)

Buscar a resposta como se fosse um nmero em um range!

[1, 2, 3, 4, 5, 6, ..., max_piles]

resposta est aqui em algum lugar

TASKS

from typing import List

def minEatingSpeed(piles: List[int], h: int) -> int:

"""

Encontra mnima velocidade para comer todas as bananas em h horas.

Estratgia: Binary Search na velocidade (1 at $\text{max}(\text{piles})$)

Args:

piles: Lista de inteiros, n pilhas

h: Inteiro, horas disponveis

Returns:

int: Mnima velocidade em bananas/hora

Time: $O(n \log(\text{max}(\text{piles})))$

- $\log(\text{max})$ iteraes de binary search

- n para calcular hours_needed a cada iterao

Space: $O(1)$

Example:

>>> minEatingSpeed([3, 6, 7, 11], 8)

4 # Com velocidade 4: $1+2+2+3 = 8$ horas (perfeit!)

"""

def hours_needed(speed: int) -> int:

"""

Calcula TOTAL de horas necessrias para comer todas as pilhas com uma velocidade k.

Para cada pilha, calcula: $\text{ceil}(\text{pile} / \text{speed}) = (\text{pile} + \text{speed} - 1) // \text{speed}$

"""

total_hours = 0

for pile in piles:

Horas para essa pilha: arredonda para CIMA

$(\text{pile} + \text{speed} - 1) // \text{speed}$ equivalente a $\text{ceil}(\text{pile} / \text{speed})$

hours_for_this_pile = $(\text{pile} + \text{speed} - 1) // \text{speed}$

total_hours += hours_for_this_pile

return total_hours

Binary Search

left = 1 # Velocidade MNIMA

right = $\text{max}(\text{piles})$ # Velocidade MXIMA

Invariante: resposta est em [left, right]

while left < right:

mid = $(\text{left} + \text{right}) // 2$

Pergunta: Com velocidade mid, consigo terminar em h horas?

if hours_needed(mid) <= h:

SIM! Ento posso tentar MAIS DEVAGAR

A resposta est em [left, mid]

```

right = mid
else:
# NO! Preciso comer MAIS RPIDO
# A resposta est em [mid+1, right]
left = mid + 1

# Agora left == right e ambos apontam para a resposta!
return left

```

TESTS

```

if __name__ == "__main__":
# Exemplo da descrio
print("Test 1: piles=[3,6,7,11], h=8")
result = minEatingSpeed([3, 6, 7, 11], 8)
print(f" Velocidade mnima: {result}")
print(f" Horas needed: {(3 + result - 1) // result + (6 + result - 1) // result + (7 + result - 1) // result + (11 + result - 1) // result}")

```

assert

```

# Edge: apenas 1 pilha
print("\nTest 2: piles=[1000000000], h=2")
result = minEatingSpeed([1000000000], 2)
print(f" Velocidade mnima: {result}")
assert result == 500000000, f"Esperava 500000000, got {result}!"

```

```

# Edge: velocidade mxima necessaria
print("\nTest 3: piles=[1,1,1,1], h=4")
result = minEatingSpeed([1, 1, 1, 1], 4)
print(f" Velocidade mnima: {result}")
assert result == 1, "Esperava 1!"

```

```

# Edge: uma hora, comer tudo em 1 hora
print("\nTest 4: piles=[312884132], h=968709470")
result = minEatingSpeed([312884132], 968709470)
print(f" Velocidade mnima: {result}")
assert result == 1, "Esperava 1!"

print("\n Todos os testes passaram!")

```

Entrada: piles = [3, 6, 7, 11], h = 8

Binary Search Loop:

```

Iterao 1:
left=1, right=11
mid = (1+11)//2 = 6
hours_needed(6) = ceil(3/6) + ceil(6/6) + ceil(7/6) + ceil(11/6)
= 1 + 1 + 2 + 2 = 6 <= 8? SIM!
Podemos ir mais devagar: right = 6

```

```

Iterao 2:
left=1, right=6
mid = (1+6)//2 = 3
hours_needed(3) = ceil(3/3) + ceil(6/3) + ceil(7/3) + ceil(11/3)
= 1 + 2 + 3 + 4 = 10 <= 8? NO!
Precisa ser mais rpido: left = 4

```

```

Iterao 3:
left=4, right=6
mid = (4+6)//2 = 5
hours_needed(5) = ceil(3/5) + ceil(6/5) + ceil(7/5) + ceil(11/5)
= 1 + 2 + 2 + 3 = 8 <= 8? SIM!
Podemos ir mais devagar: right = 5

```

```

Iterao 4:
left=4, right=5
mid = (4+5)//2 = 4
hours_needed(4) = ceil(3/4) + ceil(6/4) + ceil(7/4) + ceil(11/4)
= 1 + 2 + 2 + 3 = 8 <= 8? SIM!
Podemos ir mais devagar: right = 4

```

```

Iterao 5:
left=4, right=4
left == right PARE!

```

Resposta: 4

Hours Needed vs Speed

Horas

28 | (k=1: lento)

25 |

20 | (k=2: mais rpido)

15 |

10 | (k=3)

8 | (k=4,5,6: resposta k=4)

| (k=7+: ainda mais rpido, mas k=4 mnimo!)

k (velocidade)

1 2 3 4 5 6 7 8

Propriedade: SEMPRE DECRESCENTE (monotnico!)

Logo: Binary Search funciona!

Daily Temperatures (Monotonic Stack)

****SPEC:****

- Input: `temperatures: List[int]` (temperaturas dirias)

- Output: `List[int]` (dias at prxima temperatura MAIOR)

- Edge: `[1]`, `[1,1,1,1]` (sem prxima maior), `[30,40,50,60]` (j crescente)

- Propriedade: ****Vizinhana/Padro**** (prximo elemento maior)

****PLAN:****

- Fora bruta: Para cada dia, procura adiante $O(n)$

- Monotonic Stack: Mantm stack de ndices em ordem decrescente $O(n)$ ****MELHOR****

****Intuiio:****

Quando temperatura AUMENTA, sabemos a resposta para dias anteriores!

Exemplo: [73, 74, 75, 71, 69, 72, 76, 73]

dia 0 (73): Stack vazio, empilha

dia 1 (74): 74 > 73? SIM! Pop 73, resposta[0] = 1-0 = 1

Empilha 74

dia 2 (75): 75 > 74? SIM! Pop 74, resposta[1] = 2-1 = 1

Empilha 75

dia 5 (72): 72 > 71? SIM! Pop 71, resposta[3] = 5-3 = 2

72 > 75? NO, empilha 72

dia 6 (76): 76 > 72? SIM! Pop 72, resposta[5] = 6-5 = 1

76 > 75? SIM! Pop 75, resposta[2] = 6-2 = 4

Empilha 76

Resultado: [1, 1, 4, 2, 1, 1, 0, 0]

****CDIGO:****

```
def dailyTemperatures(temps: List[int]) -> List[int]:
```

```
    """Monotonic Stack (decrescente): mantm ndices de temperaturas."""
```

```
    n = len(temps)
```

```
    result = [0] * n
```

```
    stack = [] # Guarda NDICES, mantendo temperaturas em ordem DECRESCENTE
```

```
    for i, temp in enumerate(temps):
```

```
        # Enquanto temperatura atual > topo do stack
```

```
        while stack and temp > temps[stack[-1]]:
```

```
            j = stack.pop()
```

```
            result[j] = i - j # Encontrou a resposta!
```

```
        # Empilha ndice atual
```

```
        stack.append(i)
```

```
    return result
```

Tests

```
assert dailyTemperatures([73, 74, 75, 71, 69, 72, 76, 73]) == [1, 1, 4, 2, 1, 1, 0, 0]
```

```
assert dailyTemperatures([30, 60, 90]) == [1, 1, 0]
```

```
assert dailyTemperatures([89, 62, 70, 58, 47, 47, 46, 76, 100, 70]) == [8, 1, 5, 3, 8, 4, 3, 2, 0, 0]
```

```
print(" Daily Temperatures Passed")
```

****Trade-off:****

- Fora bruta $O(n)$ correto mas lento

assert

- Monotonic Stack $O(n)$ elegante: cada elemento entra/sai da stack UMA VEZ

****Complexidade:****

- Time: $O(n)$ - cada índice entra e sai UMA VEZ

- Space: $O(n)$ - stack pode ter até n elementos no pior caso

Kahn's Algorithm (Topological Sort)

****SPEC:****

- Input: `edges: List[List[int]]`, `n: int` (DAG - Directed Acyclic Graph)

- Output: `List[int]` (ordem topológica) ou detectar ciclo

- Propriedade: ****Dependências**** - se há ciclo, retorna []

- Exemplo: Course Prerequisites, Task Scheduling

****PLAN:****

- DFS com stack: Explorar profundidade $O(V+E)$

- Kahn's BFS: In-degree + queue $O(V+E)$ ****MELHOR para detectar ciclos****

****Intuição:****

1. Calcula in-degree (quantas arestas entram) de cada n

2. Começa com n s que tm in-degree = 0 (sem dependências)

3. Processa n : adiciona resposta, reduz in-degree dos vizinhos

4. Se conseguir processar V n s topológica válida

Se não ciclo detectado!

****CÓDIGO:****

```
def canFinish(n: int, edges: List[List[int]]) -> bool:
```

```
    """Kahn: detecta ciclo em DAG."""
```

```
    # Construir grafo e in-degree
```

```
    graph = [[] for _ in range(n)]
```

```
    in_degree = [0] * n
```

```
    for course, prereq in edges:
```

```
        graph[prereq].append(course)
```

```
        in_degree[course] += 1
```

```
    # Fila com  $n$ s que tm in-degree = 0
```

```
    queue = [i for i in range(n) if in_degree[i] == 0]
```

```
    count = 0
```

```
    while queue:
```

```
        course = queue.pop(0)
```

```
        count += 1
```

```
    # Reduz in-degree dos vizinhos
```

```
    for next_course in graph[course]:
```

```
        in_degree[next_course] -= 1
```

```
        if in_degree[next_course] == 0:
```

```
            queue.append(next_course)
```

```
    # Se processou todos os cursos, não tem ciclo
```

```
    return count == n
```

```
def topologicalSort(n: int, edges: List[List[int]]) -> List[int]:
```

```
    """Kahn: retorna ordem topológica ou [] se tem ciclo."""
```

```
    graph = [[] for _ in range(n)]
```

```
    in_degree = [0] * n
```

```
    for u, v in edges:
```

```
        graph[u].append(v)
```

```
        in_degree[v] += 1
```

```
    queue = [i for i in range(n) if in_degree[i] == 0]
```

```
    result = []
```

```
    while queue:
```

```
        node = queue.pop(0)
```

```
        result.append(node)
```

```
    for neighbor in graph[node]:
```

```
        in_degree[neighbor] -= 1
```

```
        if in_degree[neighbor] == 0:
```

```
            queue.append(neighbor)
```

```
    # Se não processou todos, tem ciclo
```

```
    return result if len(result) == n else []
```

Tests

DAG vlido: 0 1 2

```
assert canFinish(3, [[1, 0], [2, 1]]) == True
```

Ciclo: 0 1 0

```
assert canFinish(2, [[1, 0], [0, 1]]) == False
```

Sem arestas

```
assert canFinish(3, []) == True
```

Topological sort

```
result = topologicalSort(4, [[1, 0], [2, 0], [3, 1], [3, 2]])
```

```
assert len(result) == 4 # Todos processados
```

```
print(" Kahn's Algorithm / Topological Sort Passed")
```

****Complexidade:****

- Time: $O(V + E)$ - visita cada n e aresta uma vez

- Space: $O(V + E)$ - grafo + queue

Dijkstra's Algorithm (Shortest Path)

****SPEC:****

- Input: `graph: List[List[Tuple]]` (grafo com pesos), `start: int`

- Output: `List[int]` (distncia mnima de cada n at start)

- Propriedade: ****Grafos ponderados**** - sem pesos negativos!

- Padro: Network Delay Time, Path With Minimum Effort

****PLAN:****

- BFS simples: No funciona com pesos

- Dijkstra: Greedy com Priority Queue $O((V+E) \log V)$ ****MELHOR****

****Intuiio:****

1. Comea no n start com distncia 0

2. Sempre processa o n com MENOR distncia ainda no visitado

3. Atualiza distncias dos vizinhos: $\text{dist}[\text{neighbor}] = \min(\text{dist}[\text{neighbor}], \text{dist}[\text{current}] + \text{weight})$

****CDIGO:****

```
import heapq
```

```
from typing import List, Tuple
```

```
def dijkstra(n: int, edges: List[List[int]], start: int) -> List[int]:
```

```
    """Dijkstra com Priority Queue: menor distncia primeiro."""
```

```
    # Construir grafo
```

```
    graph = [[] for _ in range(n)]
```

```
    for u, v, weight in edges:
```

```
        graph[u].append((v, weight))
```

```
        graph[v].append((u, weight)) # Se no-dirigido
```

```
    # Inicializa distncias
```

```
    dist = [float('inf')] * n
```

```
    dist[start] = 0
```

```
    visited = set()
```

```
    # Priority queue: (distncia, n)
```

```
    pq = [(0, start)]
```

```
    while pq:
```

```
        d, u = heapq.heappop(pq)
```

```
        # J visitado
```

```
        if u in visited:
```

```
            continue
```

```

visited.add(u)
# Relaxa arestas
for v, weight in graph[u]:
    if v not in visited:
        new_dist = dist[u] + weight
        if new_dist < dist[v]:
            dist[v] = new_dist
        heapq.heappush(pq, (new_dist, v))
return dist

def networkDelayTime(times: List[List[int]], n: int, k: int) -> int:
    """Dijkstra: tempo para todos os ns receberem sinal."""
    # Grafo: u v com weight
    graph = [[] for _ in range(n + 1)]
    for u, v, time in times:
        graph[u].append((v, time))
    # Dijkstra do n k
    dist = [float('inf')] * (n + 1)
    dist[k] = 0
    visited = set()
    pq = [(0, k)]
    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        for v, weight in graph[u]:
            if v not in visited:
                new_dist = dist[u] + weight
                if new_dist < dist[v]:
                    dist[v] = new_dist
                heapq.heappush(pq, (new_dist, v))
    # Distancia mxima (tempo para todos receberem)
    max_time = max(dist[1:n+1])
    return max_time if max_time != float('inf') else -1

```

Tests

Exemplo: Sinal de 1 para 3 ns, k=1

Edges: 12 (1), 13 (4), 23 (2)

Path: 13 diretamente (4) ou 123 (3)

```
assert networkDelayTime([[2, 1, 1], [2, 3, 1], [3, 4, 1]], 4, 1) == 4
```

Caminho direto

```
assert networkDelayTime([[1, 2, 1]], 2, 1) == 1
```

Mltiplos caminhos

```
assert networkDelayTime([[1, 2, 1], [1, 3, 4], [2, 3, 2]], 3, 1) == 4
```

```
print(" Dijkstra's Algorithm Passed")
```

****Complexidade:****

- Time: $O((V + E) \log V)$ com Priority Queue

- Space: $O(V + E)$ - grafo + estruturas

```
---
```

Merge Intervals (Interval Pattern)

****SPEC:****

- Input: `intervals: List[List[int]]` (no-ordenados, sobrepostos)
- Output: `List[List[int]]` (intervalos mesclados)
- Propriedade: ****Ordenao + Varredura**** (sort depois merge)
- Exemplo: `[[1,3],[2,6],[8,10],[15,18]]` `[[1,6],[8,10],[15,18]]`

****PLAN:****

- Fora bruta: Comparar todos com todos $O(n)$
- Sort + Merge: Ordena por start, depois varre uma vez $O(n \log n)$ ****MELHOR****

****Intui o:****

1. Sort intervalos por start
2. Varre: se `current.start <= last.end` merge (`end = max(end)`)
Seno novo intervalo

****CDIGO:****

```
def merge(intervals: List[List[int]]) -> List[List[int]]:
    """Merge intervals: sort + varredura."""
    if not intervals:
        return []

    # Sort por start (Python sorts por primeiro elemento por padro)
    intervals.sort()

    result = [intervals[0]]
    for current in intervals[1:]:
        last = result[-1]
        # Sobrepe? Merge
        if current[0] <= last[1]:
            result[-1] = [last[0], max(last[1], current[1])]
        else:
            # Novo intervalo
            result.append(current)
    return result
```

Tests

```
assert merge([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
assert merge([[1, 4], [4, 5]]) == [[1, 5]]
assert merge([[1, 5]]) == [[1, 5]]
assert merge([]) == []
assert merge([[1, 3], [4, 6]]) == [[1, 3], [4, 6]]
print(" Merge Intervals Passed")
```

****Complexidade:****

- Time: $O(n \log n)$ - sorting dominante
- Space: $O(1)$ - no conta output

Insert Interval (Interval Pattern Avanado)

****SPEC:****

- Input: `intervals: List[List[int]]` (j ordenados!), `newInterval: List[int]`
- Output: `List[List[int]]` (intervals + newInterval mesclado)
- Propriedade: ****J ordenado**** - deve permanecer ordenado
- Tricky: No pode fazer sort novamente!

****PLAN:****

- Merge simples (sort): $O(n \log n)$ (no eficiente)
- 3-pass: Before + Merge + After $O(n)$ ****MELHOR****

****Intui o:****

- Pass 1: Adiciona todos intervalos que terminam ANTES do new start
- Pass 2: Mescla TODOS que sobrepem
- Pass 3: Adiciona todos que comeam DEPOIS do new end

****CDIGO:****

```
def insert_clean(intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
```

"""Inse

```

result = []
new_start, new_end = newInterval
# Adiciona intervals que terminam antes do novo
i = 0
while i < len(intervals) and intervals[i][1] < new_start:
    result.append(intervals[i])
    i += 1
# Mescla todos que sobrepem
while i < len(intervals) and intervals[i][0] <= new_end:
    new_start = min(new_start, intervals[i][0])
    new_end = max(new_end, intervals[i][1])
    i += 1
result.append([new_start, new_end])
# Adiciona rest que no sobrepe
while i < len(intervals):
    result.append(intervals[i])
    i += 1
return result

```

Tests

```

assert insert_clean([[1, 5]], [2, 3]) == [[1, 5]] # Dentro
assert insert_clean([[1, 5]], [2, 7]) == [[1, 7]] # Sobrepe direita
assert insert_clean([[1, 5]], [0, 0]) == [[0, 0], [1, 5]] # Antes
assert insert_clean([[1, 2], [3, 5], [6, 7]], [5, 6]) == [[1, 2], [3, 7]]
assert insert_clean([], [5, 7]) == [[5, 7]] # Vazio
print(" Insert Interval Passed")

**Complexidade:**
- Time: O(n) - uma ou duas varridas
- Space: O(1) - no conta output
---
```

Meeting Rooms II (Interval Pattern com Heap)

```

**SPEC:**
- Input: `intervals: List[List[int]]` (horrios de reunies)
- Output: `int` (mnimo salas necessrias)
- Padro: **Evento + Varredura** (incio e fim de reunio)
- Tricky: Pode reusar sala se uma termina quando outra comea

**PLAN:**
- Fora bruta: Comparar todos com todos O(n)
- Sort + Min Heap: Ordena starts, usa heap para tracks fins O(n log n) **MELHOR**

**Intuiio:**
1. Sort tudo: [start, "begin"], [end, "end"]
2. Varre:
- Incio adiciona sala (heap++)
- Fim libera sala (heap--)
3. Mximo tamanho da heap = salas necessrias

**CDIGO:**
def minMeetingRooms(intervals: List[List[int]]) -> int:
    """Min rooms: sort + min heap para track de sadas."""
    if not intervals:
        return 0
    # Cria lista de eventos: (tempo, tipo)
    # tipo: 0 = sada (deve processar primeiro), 1 = entrada
    events = []
    for start, end in intervals:
        events.append((start, 1)) # Incio
        events.append((end, 0)) # Fim
    # Sort: por tempo, depois por tipo (sada antes de entrada no mesmo tempo)
    events.sort()
    rooms_needed = 0
    rooms_in_use = 0

```

```

for time, event_type in events:
    if event_type == 1: # Reunio comea
        rooms_in_use += 1
        rooms_needed = max(rooms_needed, rooms_in_use)
    else: # Reunio termina
        rooms_in_use -= 1
return rooms_needed

def minMeetingRooms_heap(intervals: List[List[int]]) -> int:
    """Min rooms: heap verso com min-heap de sadas."""
    if not intervals:
        return 0

    # Sort por start
    intervals.sort()

    # Min-heap: guardar fins das reunies
    end_times = []
    heapq.heappush(end_times, intervals[0][1])

    for start, end in intervals[1:]:
        # Se a reunio anterior terminou, reusar sala
        if end_times[0] <= start:
            heapq.heappop(end_times)
        heapq.heappush(end_times, end)

    return len(end_times)

```

Tests

```

assert minMeetingRooms([[0, 30], [5, 10], [15, 20]]) == 2 # 0-30 + 5-10
assert minMeetingRooms([[7, 10], [2, 4]]) == 1 # Sem overlap
assert minMeetingRooms([[1, 5], [1, 6], [1, 7]]) == 3 # Todas simultneas
assert minMeetingRooms([]) == 0 # Vazio
print(" Meeting Rooms II Passed")

**Complexidade:**
- Time: O(n log n) - sorting + heap ops
- Space: O(n) - heap pode ter at n elementos

```

Padres Diversos: Referncia Rpida

Alm dos exemplos detalhados acima, aqui esto outros padres IMPORTANTES:

Trie + DFS (String Patterns)

GATILHOS: "Prefix", "Word dictionary", "autocomplete"
 Exemplo: Implement Trie, Word Search II
 Padro: Build Trie DFS com backtracking
 Complexidade: O(nmk) onde n=dict size, m=word len, k=branching

Topological Sort (DAG - Directed Acyclic Graph)

GATILHOS: "Dependncias", "Task scheduling", "Course prerequisites"
 Exemplo: Course Schedule, Task Sequencing
 Padro: Kahn's algorithm (BFS) ou DFS com visited
 Complexidade: O(V + E)

Segment Tree / Fenwick Tree (Range Queries)

GATILHOS: "Range sum", "Point update", "Mnimo/mximo em range"
 Exemplo: Range Sum Query, Count of Smaller Numbers After Self
 Padro: Build tree Query + Update
 Complexidade: O(log n) por operao

Two Pointers (String/Array)

GATILHOS: "Duas extremidades", "Contraste", "Remove duplicates"

Exemplo: Container With Most Water, Valid Palindrome, Merge Sorted

Padro: left=0, right=n-1, move baseado em condio

Complexidade: $O(n)$

Binary Search (no s em answer)

GATILHOS: "Array ordenado", "Buscar posio", "First/Last occurrence"

Exemplo: First Bad Version, Find Minimum in Rotated Array

Padro: left=0, right=n-1, mid=(left+right)//2

Complexidade: $O(\log n)$

Greedy (com prova!)

GATILHOS: "Mximo/mnimo com escolha local"

Exemplo: Jump Game, Interval Scheduling, Gas Station

CRTICO: Sempre prove ou teste exaustivamente!

Padro: Escolha local tima timo global

HashMap/HashSet (Frequency)

GATILHOS: "Contagem", "Frequencia", "Encontrar duplicatas"

Exemplo: Two Sum, Contains Duplicate, Valid Anagram

Padro: count[x] += 1 durante iterao

Complexidade: $O(n)$ average, $O(n)$ worst case (colises)

Heap / Priority Queue (Top K)

GATILHOS: "Top K", "Kth elemento", "Merge K lists"

Exemplo: Kth Largest Element, Top K Frequent Words

Padro: heappush() / heappop()

Complexidade: $O(n \log k)$

Dynamic Programming (Subproblems)

GATILHOS: "Mnimo/mximo", "Contar formas", "possvel?"

Exemplo: Coin Change, House Robber, LIS

Padro: $dp[i] = f(dp[i-1], dp[i-2], \dots)$

Complexidade: $O(nm)$ ou $O(n)$

Backtracking (All Solutions)

GATILHOS: "Todas as combinaes", "Todas as permutaes", "Puzzles"

Exemplo: N-Queens, Permutations, Sudoku Solver

Padro: Explore Prune Backtrack

Complexidade: $O(kn!)$ onde k tamanho da soluo

Matriz de Deciso Rpida

TIPO DE PROBLEMA PADRO SUGERIDO

Array/String ordenado?

Binary Search $O(\log n)$

Precisa de todas as solues?

Backtracking / DFS $O(kn!)$

Otimizao com subestrutura tima?

Dynamic Programming $O(n)$ ou $O(n)$

Prximo elemento MAIOR/MENOR?

Monotonic Stack $O(n)$

Grafo com vizinhanas/nveis?

BFS (shortest path) $O(V+E)$

DFS (conectividade) $O(V+E)$

Topological Sort (DAG) $O(V+E)$

DSU (ciclos/componentes) $O(n)$
Substring/subarray com janela?
Sliding Window $O(n)$
Two Pointers $O(n)$
Contagem/frequencia?
HashMap $O(n)$ average
Heap para Top K $O(n \log k)$
Escolha local tima global tima?
Greedy $O(n)$ (COM PROVA!)
Funco MONOTONICA?
Binary Search on Answer $O(n \log \max)$

Workflow Final: Como Abordar Qualquer Problema

1. LER O PROBLEMA
2. SPEC: Entrada? Sada? Constraints? Edge cases?
3. FORA BRUTA: Sempre comece com brute force!
(Te ensina o que precisa otimizar)
4. RECONHECER PADRO:
 - Grafo/rvore? BFS/DFS/DSU
 - Otimizao? Funco monotnica? Binary Search
 - Otimizao? Subestrutura tima? DP
 - Otimizao? Greedy choice? Greedy (com prova!)
 - Precisa todas solues? Backtracking
 - Vizinhana/ordena local? Monotonic Stack/Two Pointers
 - Substring/janela? Sliding Window
 - Frequencia? HashMap/Heap
 - String patterns? Trie/Regex
5. PLAN: Trade-off entre tcnicas
6. CDIGO: Implementar com testes
7. VALIDAR: Edge cases + complexidade

Tree Algorithms: Traversals & Construction

Tree Traversals (DFS Variants)

```
**SPEC:**
- Input: `root: TreeNode`
- Output: `List[int]` (valores em ordem especifica)
- Padro: **In-order, Pre-order, Post-order, Level-order (BFS)**
- Propriedade: Cada tipo de traversal tem uso especifico

**Tipos de Traversal:**
1
/ \
2 3

PRE-ORDER (root, left, right): [1, 2, 3] Serialization
IN-ORDER (left, root, right): [2, 1, 3] BST sorted
POST-ORDER (left, right, root): [2, 3, 1] Delete, evaluate
LEVEL-ORDER (BFS): [1, 2, 3] Layer by layer

**CDIGO:**
from typing import Optional, List
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def preorder(root: Optional[TreeNode]) -> List[int]:
    """Pre-order: root left right."""
```

```

result = []
def dfs(node):
if not node: return
result.append(node.val)
dfs(node.left)
dfs(node.right)
dfs(root)
return result

def inorder(root: Optional[TreeNode]) -> List[int]:
"""In-order: left root right. BST sorted!"""
result = []
def dfs(node):
if not node: return
dfs(node.left)
result.append(node.val)
dfs(node.right)
dfs(root)
return result

def postorder(root: Optional[TreeNode]) -> List[int]:
"""Post-order: left right root."""
result = []
def dfs(node):
if not node: return
dfs(node.left)
dfs(node.right)
result.append(node.val)
dfs(root)
return result

def levelorder(root: Optional[TreeNode]) -> List[List[int]]:
"""Level-order: BFS, retorna por nvel."""
if not root: return []
result, queue = [], deque([root])
while queue:
level_size = len(queue)
level = []
for _ in range(level_size):
node = queue.popleft()
level.append(node.val)
if node.left: queue.append(node.left)
if node.right: queue.append(node.right)
result.append(level)
return result

```

Tests

```

root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
assert preorder(root) == [1, 2, 4, 5, 3]
assert inorder(root) == [4, 2, 5, 1, 3]
assert postorder(root) == [4, 5, 2, 3, 1]
assert levelorder(root) == [[1], [2, 3], [4, 5]]
print(" Tree Traversals Passed")

---
```

Construct Tree from Preorder & Inorder

```

**SPEC:**
- Input: `preorder: List[int]`, `inorder: List[int]`
- Output: `TreeNode` (reconstruir rvore)
- Propriedade: Pre + Inorder determinam rvore unicamente

**CDIGO:**
def buildTree(preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
"""Reconstruct tree from preorder & inorder."""
if not preorder or not inorder:
return None
root_val = preorder[0]

```

```

root = TreeNode(root_val)
root_idx = inorder.index(root_val)
root.left = buildTree(preorder[1:root_idx+1], inorder[:root_idx])
root.right = buildTree(preorder[root_idx+1:], inorder[root_idx+1:])
return root

```

Tests

```

preorder = [3, 9, 20, 15, 7]
inorder = [9, 3, 15, 20, 7]
root = buildTree(preorder, inorder)
assert inorder(root) == [9, 3, 15, 20, 7]
print(" Construct Tree Passed")
---
```

Linked List: Advanced Patterns

Merge K Sorted Lists

```

**SPEC:**
- Input: `lists: List[Optional[ListNode]]` (k listas ordenadas)
- Output: `ListNode` (uma lista ordenada)
- Padro: **Heap O(n log k)** ou **Divide & Conquer**

**CDIGO:**
class ListNode:
def __init__(self, val=0, next=None):
self.val = val
self.next = next

def mergeKLists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
"""Merge K lists com min-heap."""
if not lists: return None

heap = []
for i, node in enumerate(lists):
if node:
heapq.heappush(heap, (node.val, i, node))

dummy = ListNode(0)
current = dummy

while heap:
val, i, node = heapq.heappop(heap)
current.next = node
current = current.next
if node.next:
heapq.heappush(heap, (node.next.val, i, node.next))

return dummy.next

```

Tests

```

def create_list(arr):
if not arr: return None
head = ListNode(arr[0])
curr = head
for val in arr[1:]:
curr.next = ListNode(val)
curr = curr.next
return head

def list_to_array(node):
result = []
while node:
result.append(node.val)
node = node.next
return result

l1 = create_list([1,4,5])

```

```

l2 = create_list([1,3,4])
l3 = create_list([2,6])
result = mergeKLists([l1, l2, l3])
assert list_to_array(result) == [1,1,2,1,3,4,4,5,6]
print(" Merge K Lists Passed")
---
```

Reorder List

```

**SPEC:**
- Input: `head: ListNode`
- Output: Reordena in-place: [1,2,3,4] [1,4,2,3]
**CDIGO:**
def reorderList(head: Optional[ListNode]) -> None:
    """Reorder: divide reverse merge alternado."""
    if not head or not head.next: return

    # Encontra meio
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    # Reverse segunda metade
    second = slow.next
    slow.next = None

    def reverse(node):
        prev = None
        while node:
            next_temp = node.next
            node.next = prev
            prev = node
            node = next_temp
        return prev

    second = reverse(second)

    # Merge alternado
    first = head
    while first and second:
        first_next, second_next = first.next, second.next
        first.next = second
        second.next = first_next
        first = first_next
        second = second_next

```

Tests

```

l1 = create_list([1,2,3,4])
reorderList(l1)
assert list_to_array(l1) == [1,4,2,3]
print(" Reorder List Passed")
---
```

DP Avanado: LIS & LCS

Longest Increasing Subsequence (LIS)

```

**SPEC:**
- Input: `nums: List[int]`
- Output: `int` (comprimento LIS)
- Exemplo: [10,9,2,5,3,7,101,18] 4 ([2,3,7,101])
**CDIGO O(n):**
def lengthOfLIS(nums: List[int]) -> int:
    """LIS com DP: dp[i] = LIS terminando em i."""
    if not nums: return 0

    n = len(nums)

```

```

dp = [1] * n
for i in range(n):
    for j in range(i):
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)
return max(dp)

```

Tests

```

assert lengthOfLIS([10,9,2,5,3,7,101,18]) == 4
assert lengthOfLIS([3,10,2,1,20]) == 3
print(" LIS Passed")

**CDIGO O(n log n) com Binary Search:**
def lengthOfLIS_fast(nums: List[int]) -> int:
    """LIS com Binary Search."""
    import bisect
    tails = []

    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)

assert lengthOfLIS_fast([10,9,2,5,3,7,101,18]) == 4
print(" LIS (Fast) Passed")

---

```

Longest Common Subsequence (LCS)

```

**SPEC:**
- Input: `text1: str`, `text2: str`
- Output: `int` (comprimento LCS)
- Exemplo: "abcde" e "ace" 3

**CDIGO:**
def longestCommonSubsequence(text1: str, text2: str) -> int:
    """LCS com DP 2D."""
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

```

Tests

```

assert longestCommonSubsequence("abcde", "ace") == 3
assert longestCommonSubsequence("abc", "abc") == 3
assert longestCommonSubsequence("abc", "def") == 0
print(" LCS Passed")

---

**Criado em:** Maio 2026
**Verso:** 1.1
**Status:** Master Reference para FAANG Preparation

```